

CSE107 — Lab #8

Game Design with VT100 Library

Lab Date: 28/11/2025 — Due: 05/12/2025 08:00 am

Student: Zeynep Özge TOPTAŞ
Student ID: 240104004006
Section: 2. Section
Instructor: Zeliha ERİM

1. Objectives

This lab aimed to use the ncurses library to design and implement a basic terminal-based game called Space Invaders. I used structs for game objects (player, enemy, bullet), initialized the terminal environment using ncurses functions, and established the core game design structure (input handling, update logic, drawing/rendering, and main game loop). The goal was to transform the static terminal into a dynamic graphical interface.

2. Environment

Fill in the following ones you used in the lab and **give brief information** for each one of them.

- **Operating System: Linux**
- **IDE: VSCode / GCC**

3. Problem Solving

3.1 The lab taught how to use Terminal User Interface (TUI) concepts to build an interactive game. We learned terminal manipulation (cursor control) and implementing real-time game logic for movement, collisions, and state changes within a fixed-rate game loop.

3.2 ncurses is a C library for creating TUI applications. It makes different terminals look the same to the programmer, letting you work with simple x-y coordinates. We can use it for building interactive menus, command-line text editors, and system dashboards.

3.3

Usefulness for Games	Limitations
Instant Input: Reads keys immediately (non-blocking) for smooth control.	No Graphics: Only supports ASCII/Unicode characters.

Usefulness for Games	Limitations
Optimized Drawing: Redraws only changed parts of the screen.	Limited Colors: Basic color support compared to GUI frameworks.
	Low Performance: Can cause flickering with too many objects.

3.4 To fix the speed control problem, I used the `napms(80)` function in the main loop to pause the program for 80 milliseconds. This made the game speed stable. For Enemy Movement Sync, we moved enemies only every 3 frames (`game.frame % 3 == 0`). This kept their speed slower than the player's. Finally, Collision Detection was solved by using simple coordinate checks (x,y) between objects. When a hit occurred, we reduced `plyr->lives`.

3.5 We implemented a Multi-Shot and Permanent Speed Boost feature using the 'm' key. The Super Rocket fires four bullets at once by running a small loop in the input function. It also permanently increases `game->bullet_speed`.

3.6 We used several important functions. `initscr()` starts the terminal interface, and `endwin()` restores it. For real-time control, `cbreak()` and `nodelay()` let us read keys instantly without pausing the game. We drew objects with `mvprintw()` and updated the screen visually with `refresh()`. Finally, `napms(ms)` controlled the game speed, and C standard functions like `srand` and `time` were used for random enemy actions.

3.7 The code uses structs (like `player` and `gamestate`) to store data. The main logic runs in a while loop that constantly checks these steps: Input (`handle_input`) → Update (bullet movement, collisions) → Draw (`draw_elements`). Enemy movement is controlled by checking the frame counter (`game.frame % 3 == 0`) to manage speed. The Super Rocket mode was built into the `handle_input` function, checking the 'm' key to instantly create multiple fast bullets.

4. Labwork Evidence — Screenshots

1. **Terminal and Ncurses Initialization:** First, we set up the environment by including necessary libraries and calling the `initscr()` function. We configured the terminal with `cbreak()` and `nodelay()` for instant input and hid the cursor with `curs_set(0)`. The initial welcome screen and controls were displayed.
2. **Core Game Implementation:** We focused on implementing the main game logic: `initialize_game()` set up the enemies, and the main loop handled the functions for input, updating, and drawing. We implemented the movement logic for the player ('a' and 'd') and the enemies.
3. **Collision and Scoring:** We verified the collision detection in `update_bullets()` and `update_enemy_bullets()`. When a player bullet hit an enemy, the enemy's active flag was set to 0, and the score was increased by 10. When an enemy bullet hit the player, `plyr->lives` decreased.
4. **Super Rocket/Multi-Shot Test:** We specifically tested the enhancement feature. By pressing the 'm' key, we verified that the `game->bullet_speed` increased and that four simultaneous bullets were launched from the player's position, confirming the multi-shot functionality.
5. **Game Termination:** Finally, we confirmed the game's termination conditions: winning (all enemies destroyed) or losing (lives reaching zero or `MAX_SHOTS_ALLOWED` limit reached). The `endwin()` function was called to return the terminal to its normal state.

